

LangChain & LangGraph

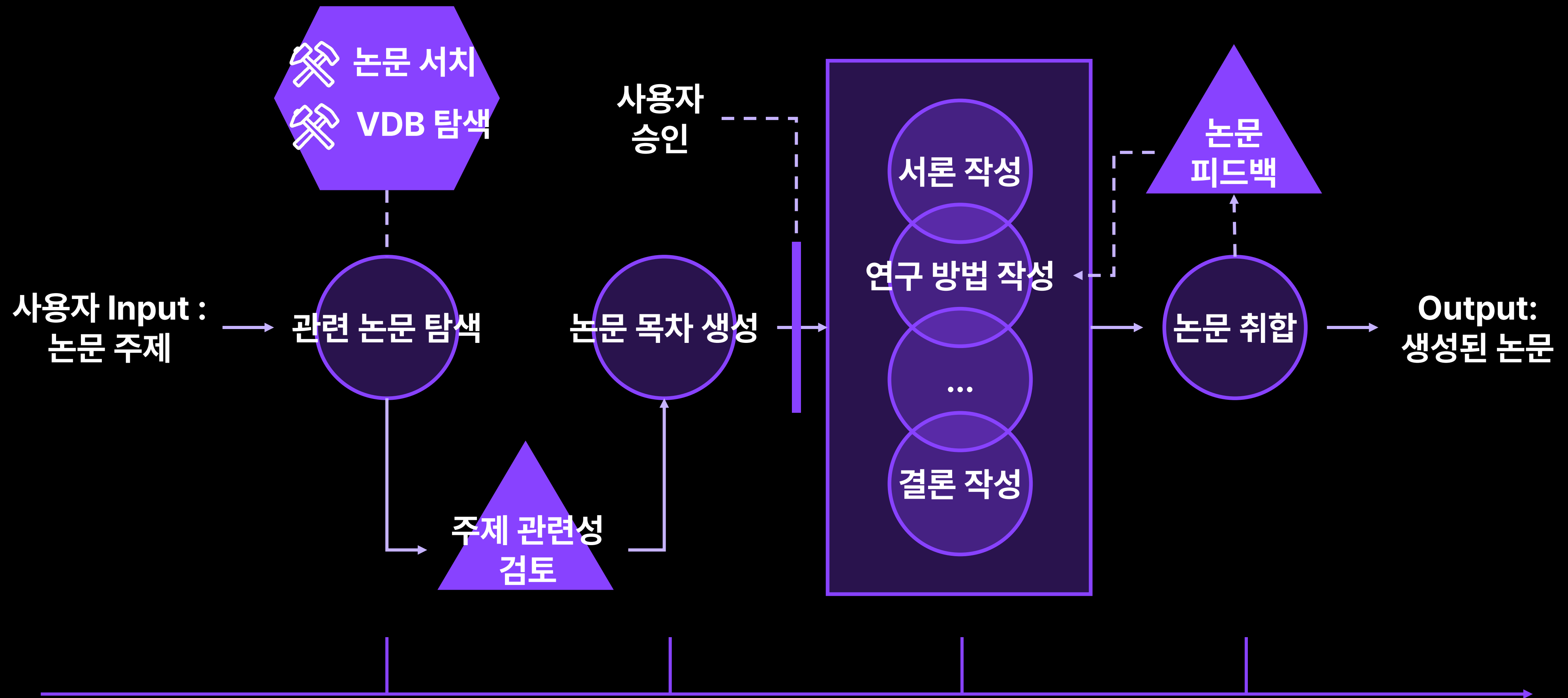
LangChain과 LangGraph

03 LangGraph – 심화

목차 03 LangGraph – 심화

1. 유연한 Agent를 만들기 위한 필수 요소
2. History 관리
3. Sub graph
4. 병렬 실행 노드

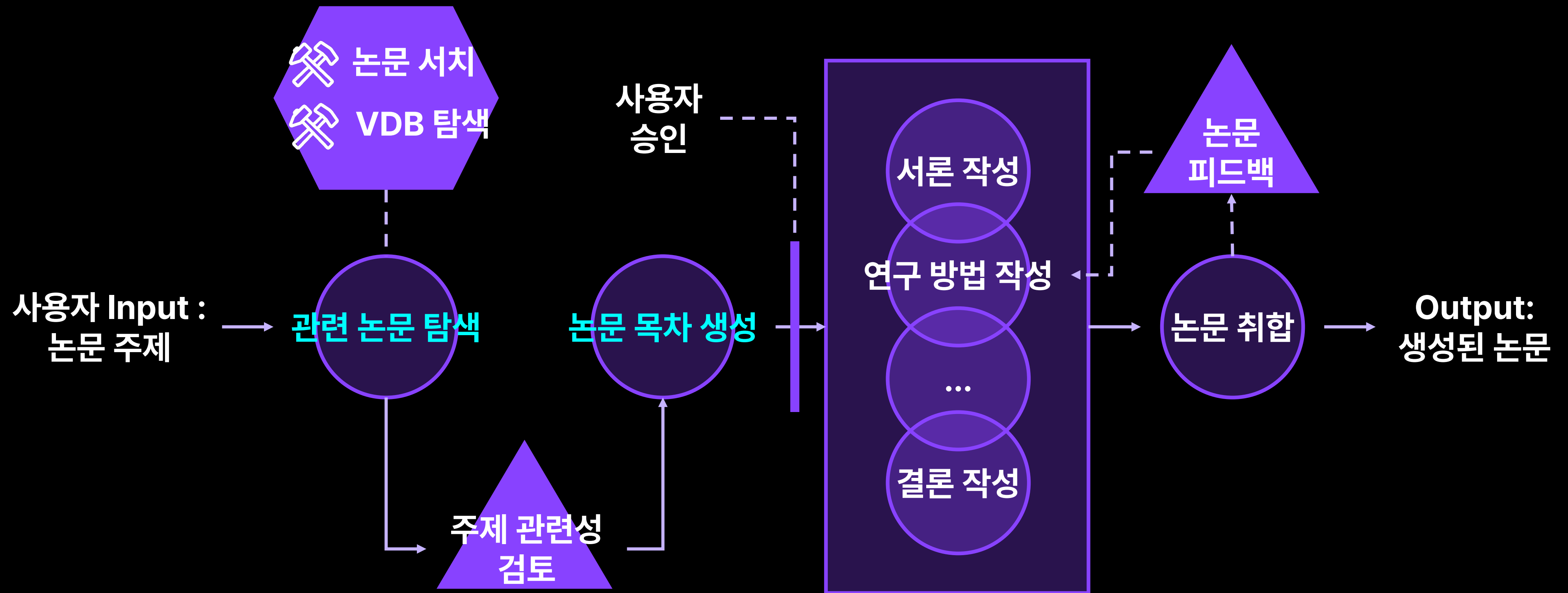
1. 유연한 Agent를 만들기 위한 필수 요소





History 관리

- Logging과는 다른 역할
- 각 노드 안에서 생성되는 LLM history 저장을 통해 다음 LLM 작업과 연결 가능
- 실행 중단 / 새로운 인풋 등의 상황에 사용



2. Sub graph 생성을 통한 분업화

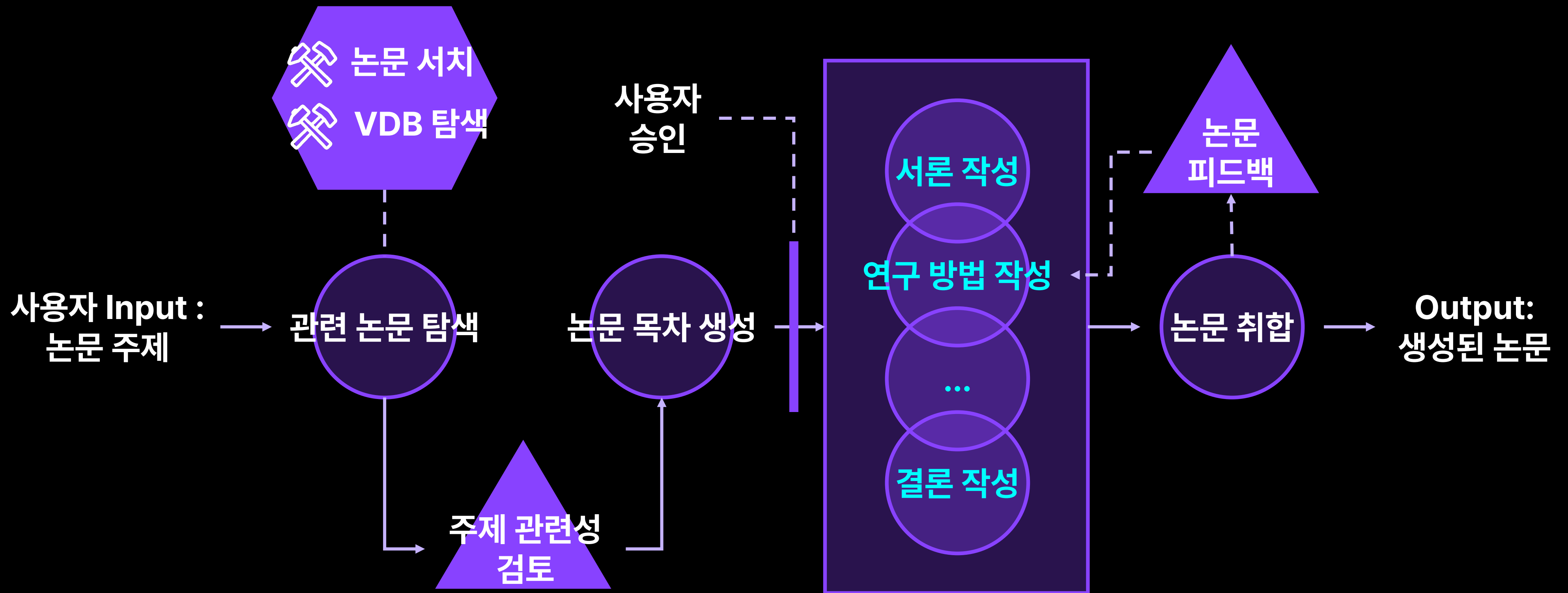


History 관리

- Logging과는 다른 역할
- 각 노드 안에서 생성되는 LLM history 저장을 통해 다음 LLM 작업과 연결 가능
- 실행 중단 / 새로운 input 등의 상황에 사용

Sub graph

- 너무 많은 기능을 하나의 그래프에 넣을 경우 구축 및 디버깅 난이도가 상승함
- Task 단위로 agent를 분리하여 하나의 '팀'을 생성하고
- 해당 팀을 '관리'하는 master agent 생성 구조 필요





History 관리

- Logging과는 다른 역할
- 각 노드 안에서 생성되는 LLM history 저장을 통해 다음 LLM 작업과 연결 가능
- 실행 중단 / 새로운 인풋 등의 상황에 사용

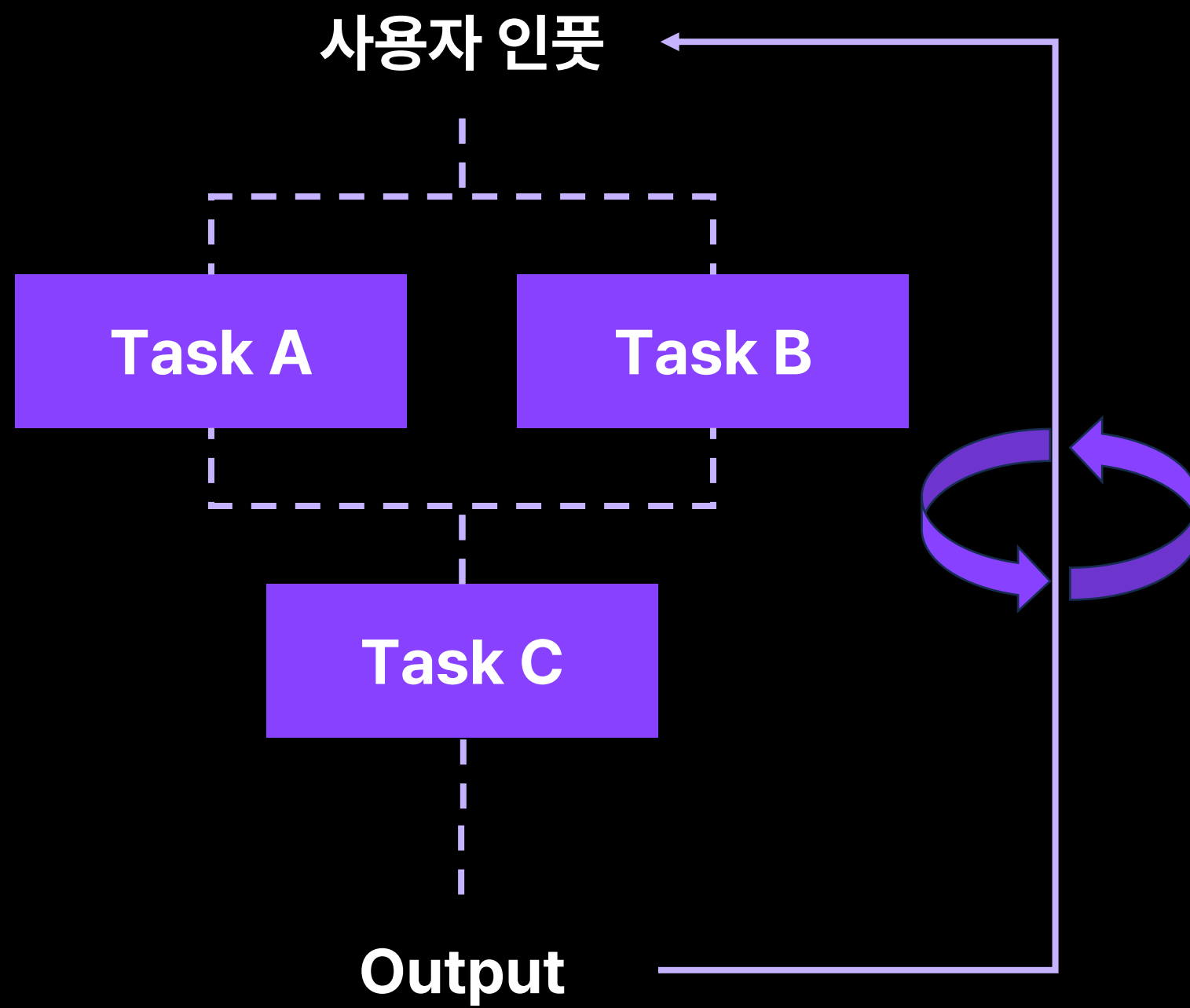
Sub graph

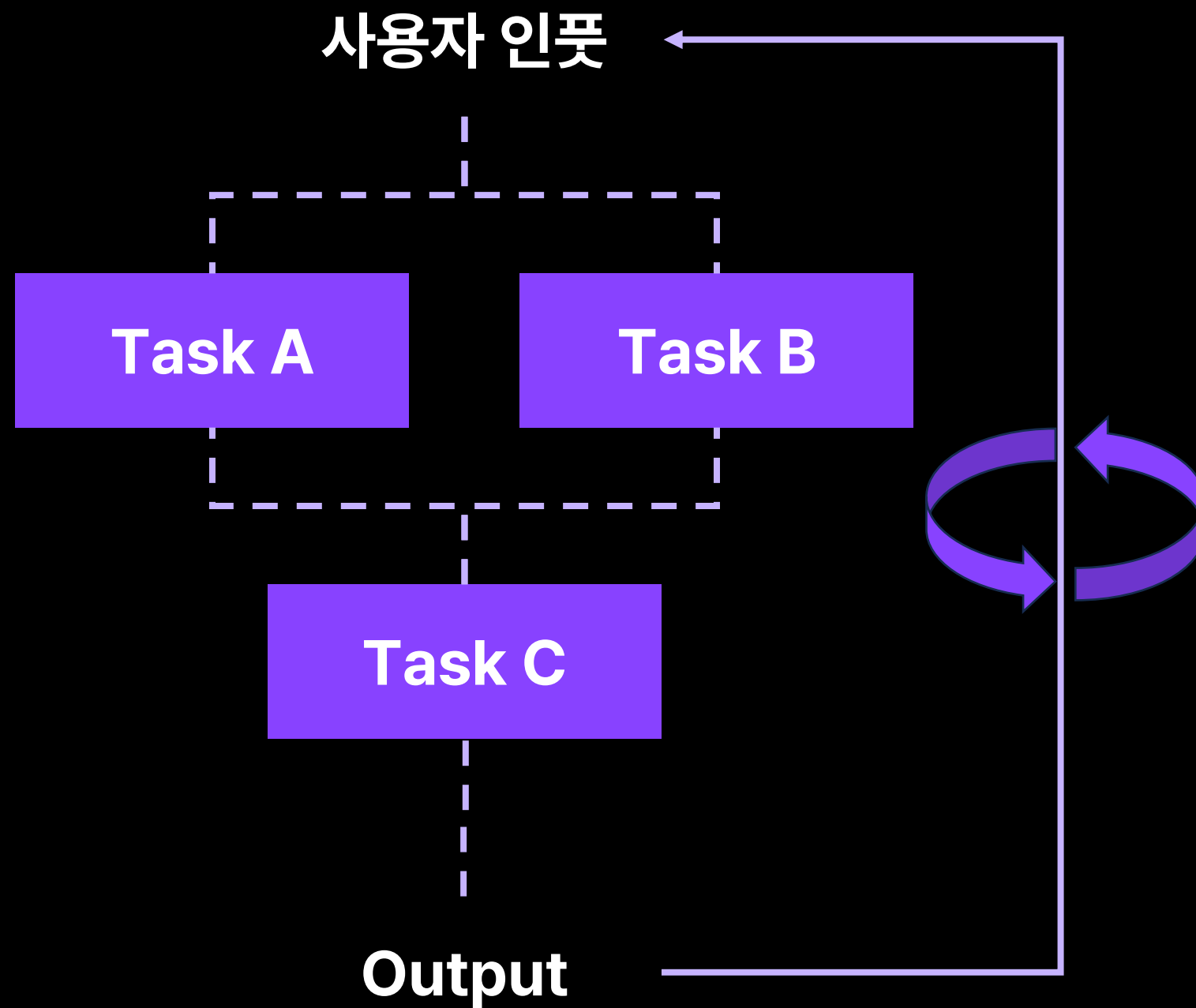
- 너무 많은 기능을 하나의 그래프에 넣을 경우 구축 및 디버깅 난이도가 상승함
- Task 단위로 agent를 분리하여 하나의 '팀'을 생성하고
- 해당 팀을 '관리'하는 master agent 생성 구조 필요

병렬 실행 노드

- 직선 형태의 Agent 구조에 병렬 구조를 더함으로써 더욱 유연한 아키텍처 구축
- 빠른 실행 속도
- 같은 task 수행은 map-reduce 노드,
- 다른 task 수행은 병렬 실행 노드 활용

2. History 관리

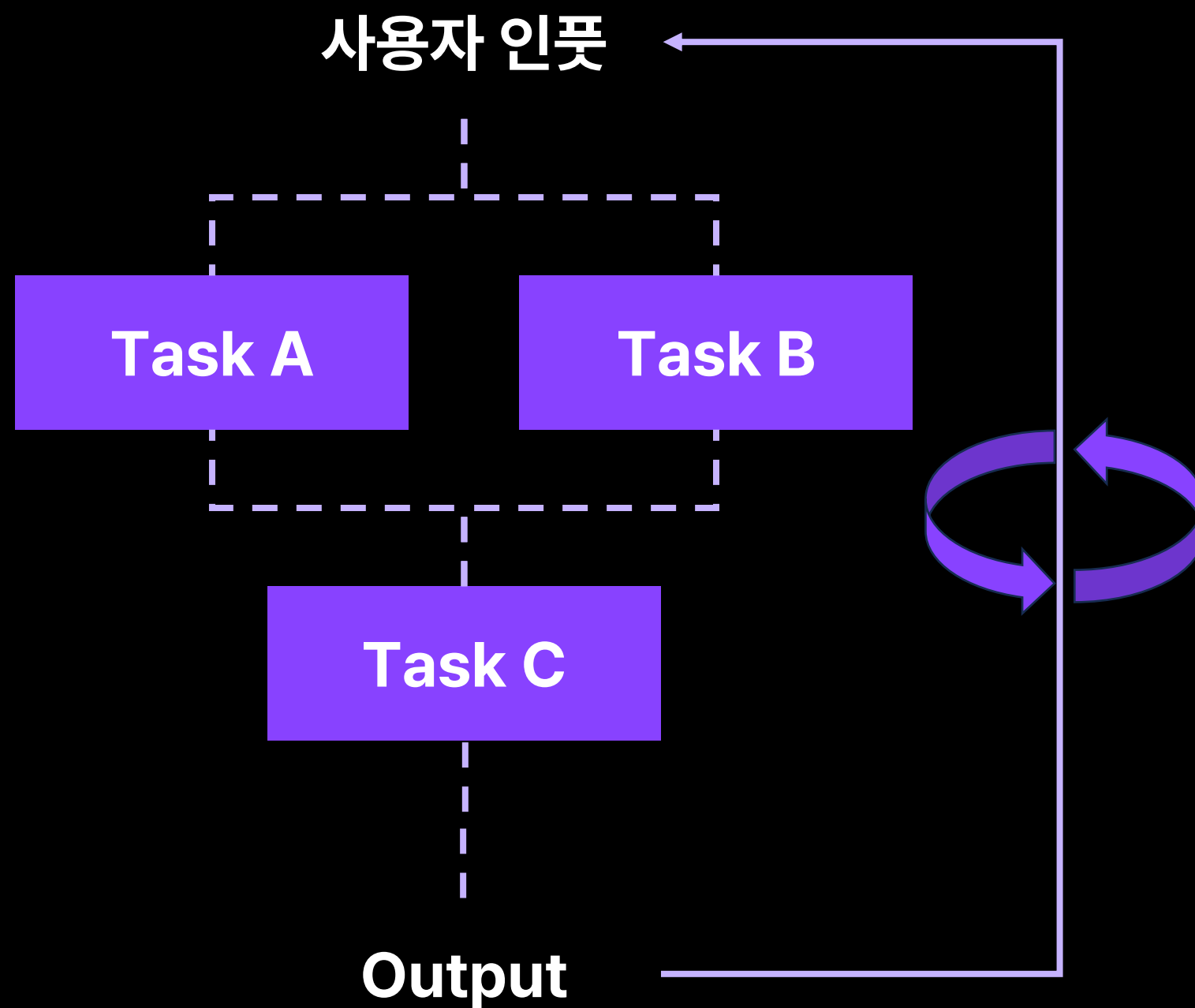




Memory

[Why]

- 사용자와의 인터랙션을 지속해야 하는 경우 존재
- 이전 대화 내역을 '재활용' 하여 비용 절감 및 퍼포먼스 향상에 기여할 수 있음



Memory

[Why]

- 사용자와의 인터랙션을 지속해야 하는 경우 존재
- 이전 대화 내역을 '재활용' 하여 비용 절감 및 퍼포먼스 향상에 기여할 수 있음

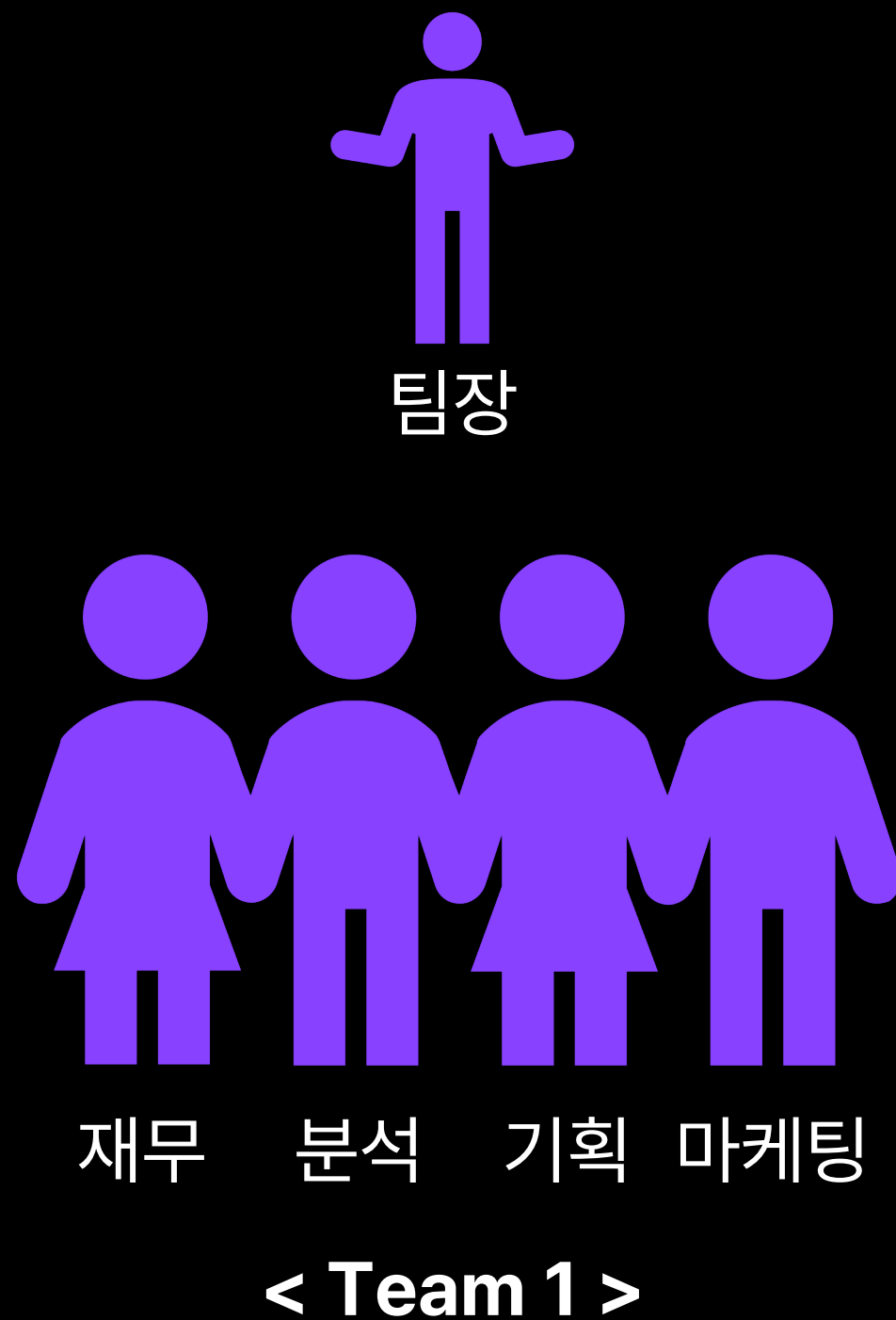
[How]

- BaseMessage를 통해 LLM의 input, output message를 관리할 수 있음
- 과거 히스토리를 인풋과 함께 제공하여 이전 "대화 내역" 참고 가능

3. Sub graph



Sub graph – what



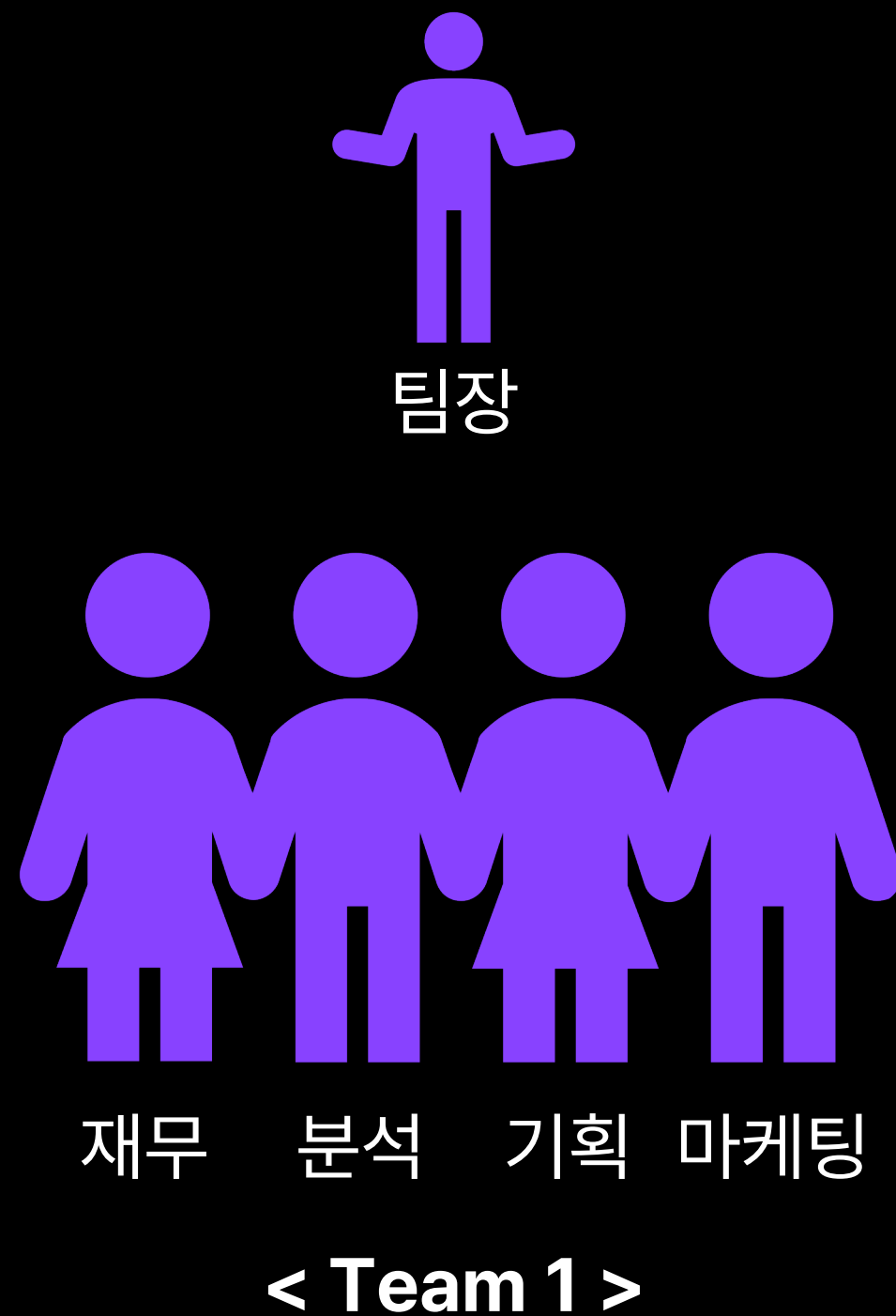
조직화 된 team

[역할 분배]

- 각자의 담당과 책임이 있다
- 개인의 성과가 모여 하나의 큰 성과를 이룬다



Sub graph - what



조직화 된 team

[역할 분배]

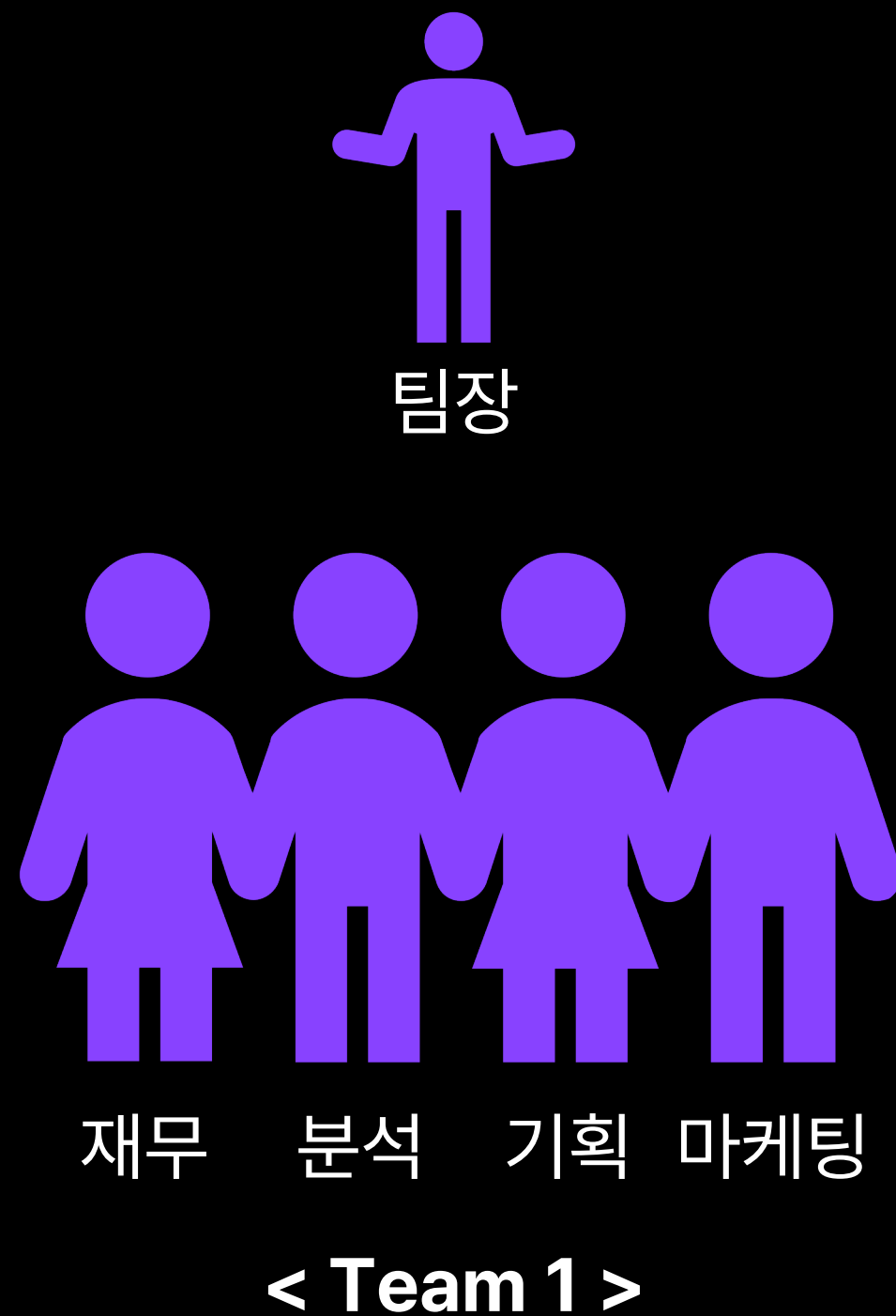
- 각자의 담당과 책임이 있다
- 개인의 성과가 모여 하나의 큰 성과를 이룬다

[상호작용]

- 팀원들끼리 상호작용 한다
- 목표를 위해 함께 움직인다



Sub graph - what



조직화 된 team

[역할 분배]

- 각자의 담당과 책임이 있다
- 개인의 성과가 모여 하나의 큰 성과를 이룬다

[상호작용]

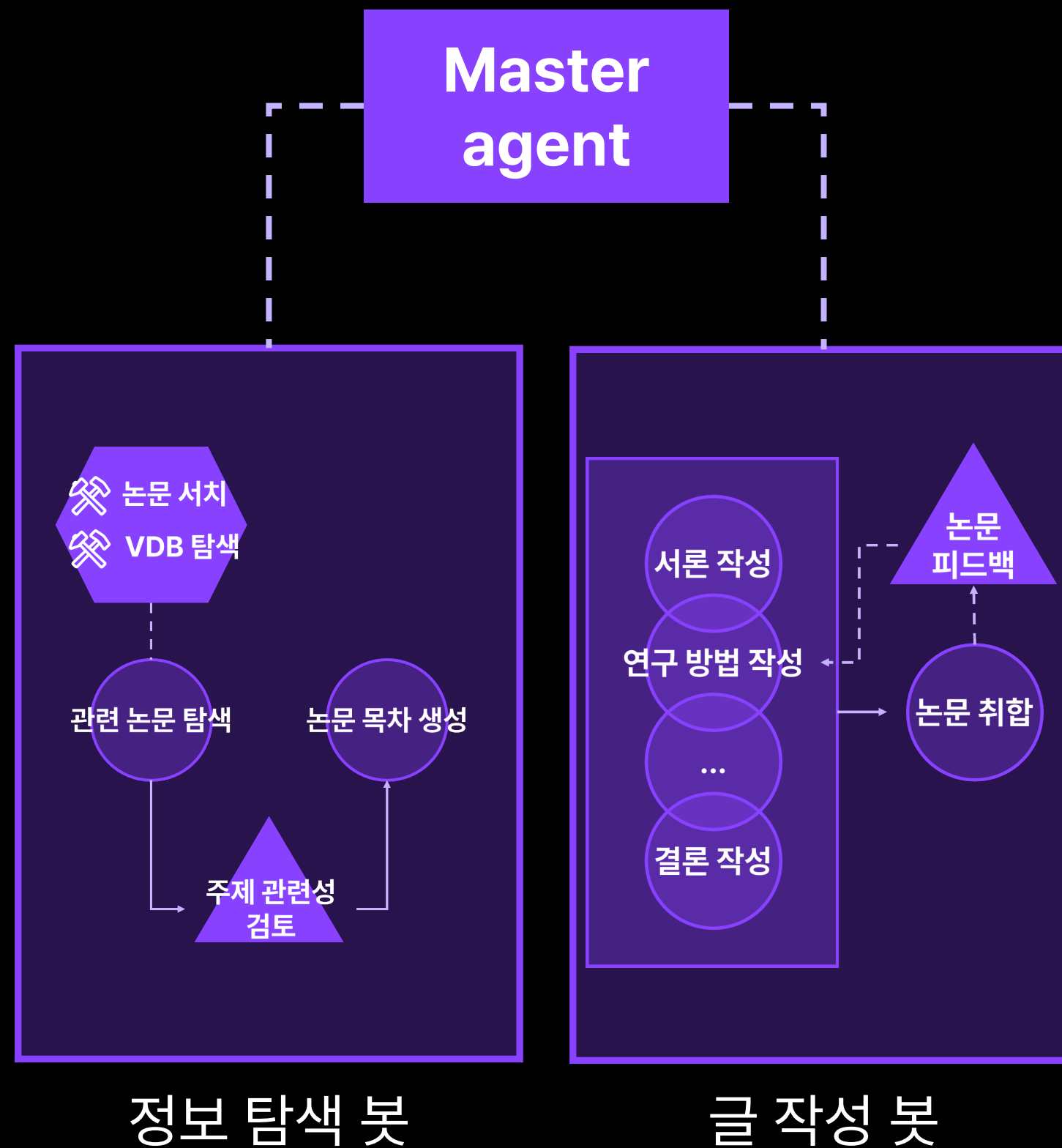
- 팀원들끼리 상호작용 한다
- 목표를 위해 함께 움직인다

[관리]

- 팀장이 방향을 정하며, 각자의 아웃풋을 검증한다



Sub graph - what



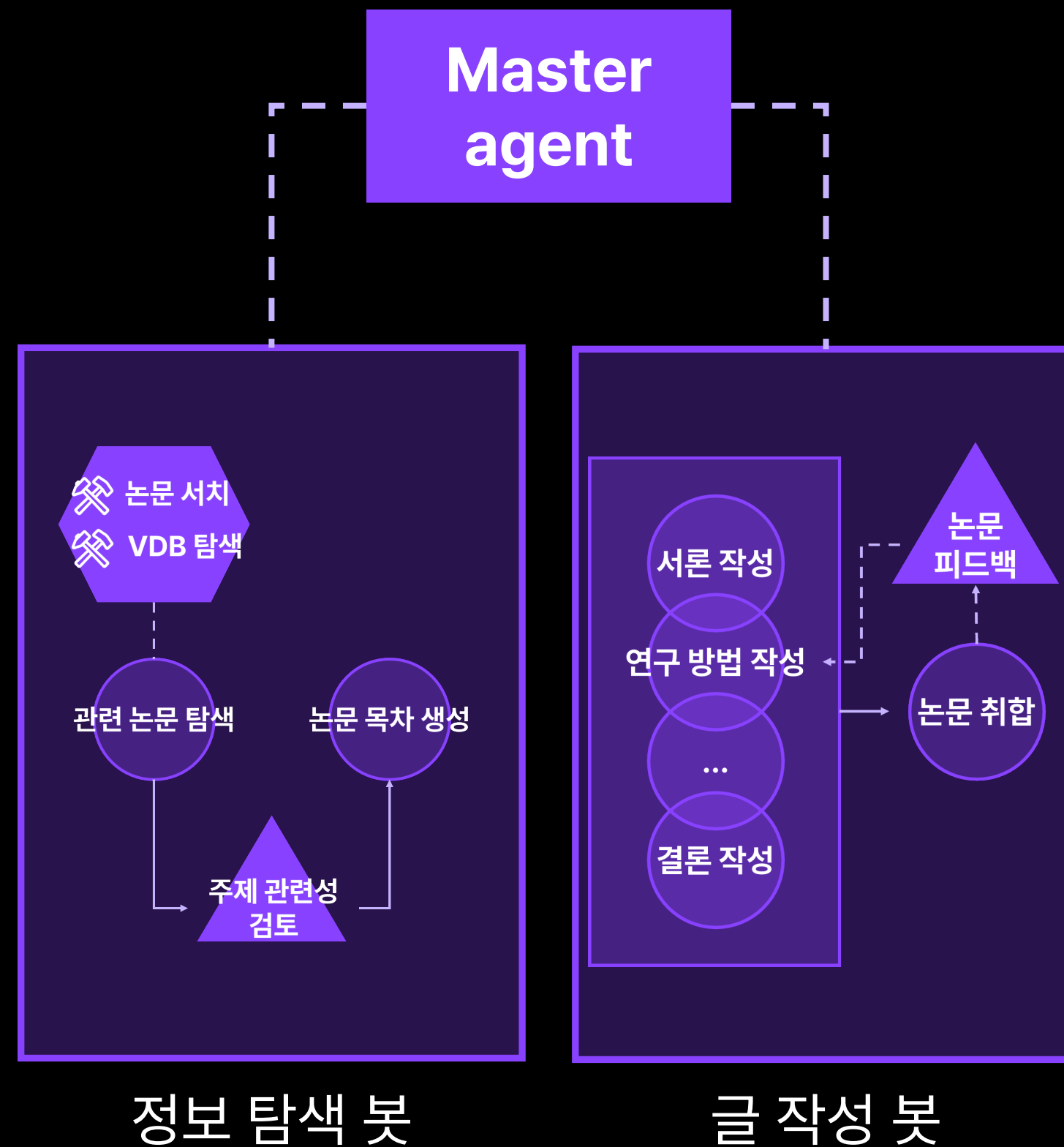
조직화 된 agent

[역할 분배]

- 각자의 담당과 책임이 있다
- 각 agent의 output이 최종 결과 생성에 기여한다



Sub graph - what



조직화 된 agent

[역할 분배]

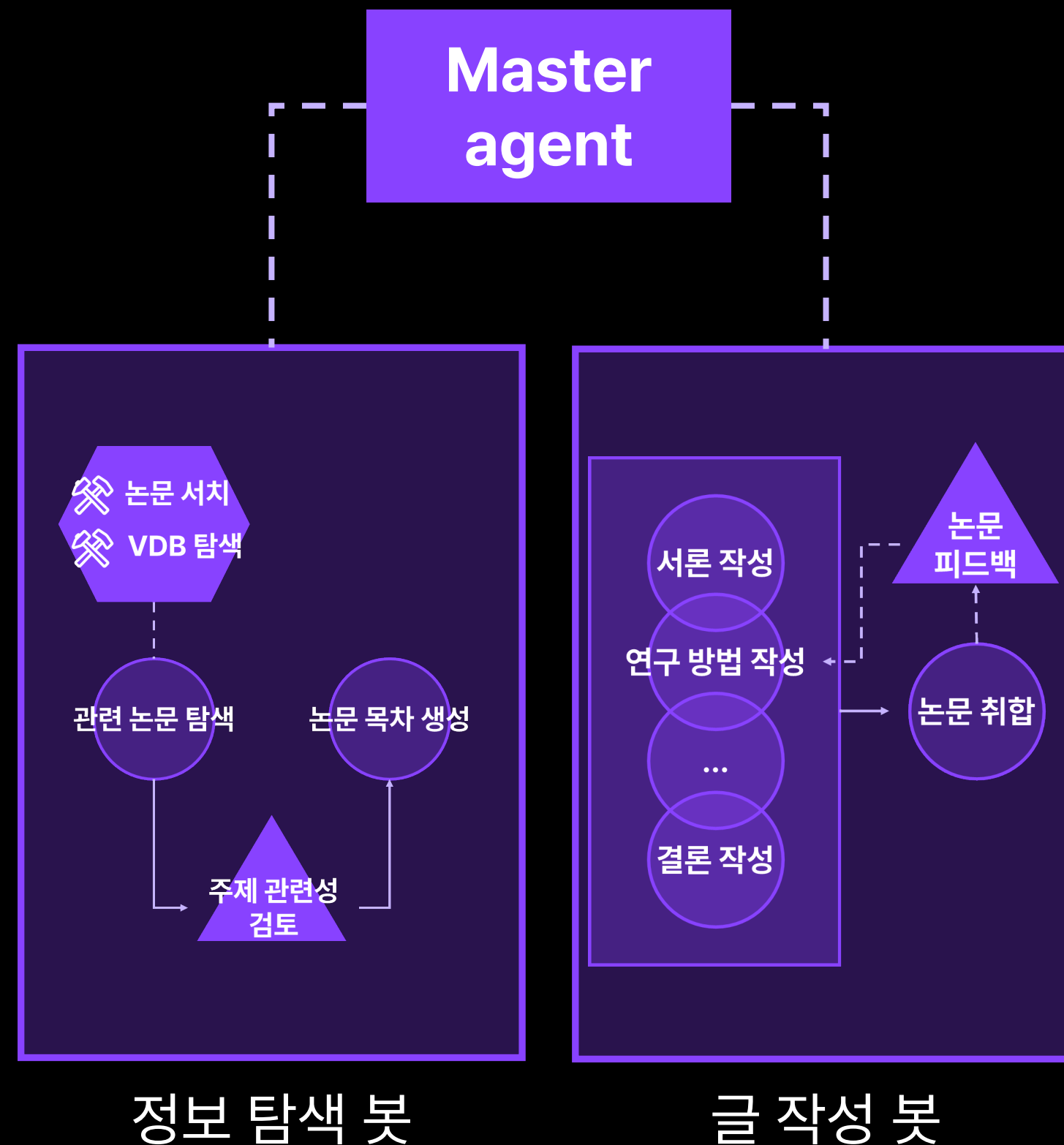
- 각자의 담당과 책임이 있다
- 각 agent의 output이 최종 결과 생성에 기여한다

[상호작용]

- Agent들끼리 상호작용 한다
- 목표를 위해 함께 움직인다



Sub graph – what



조직화 된 agent

[역할 분배]

- 각자의 담당과 책임이 있다
- 각 agent의 output이 최종 결과 생성에 기여한다

[상호작용]

- Agent들끼리 상호작용 한다
- 목표를 위해 함께 움직인다

[관리]

- Mater agent가 전체를 관장한다



Sub graph를 사용하는 이유

[Agent 구축의 용이성]

- 복잡한 문제를 해결해야 하는 경우, task를 분업화 함으로서 디버깅 및 구축이 용이해 집니다

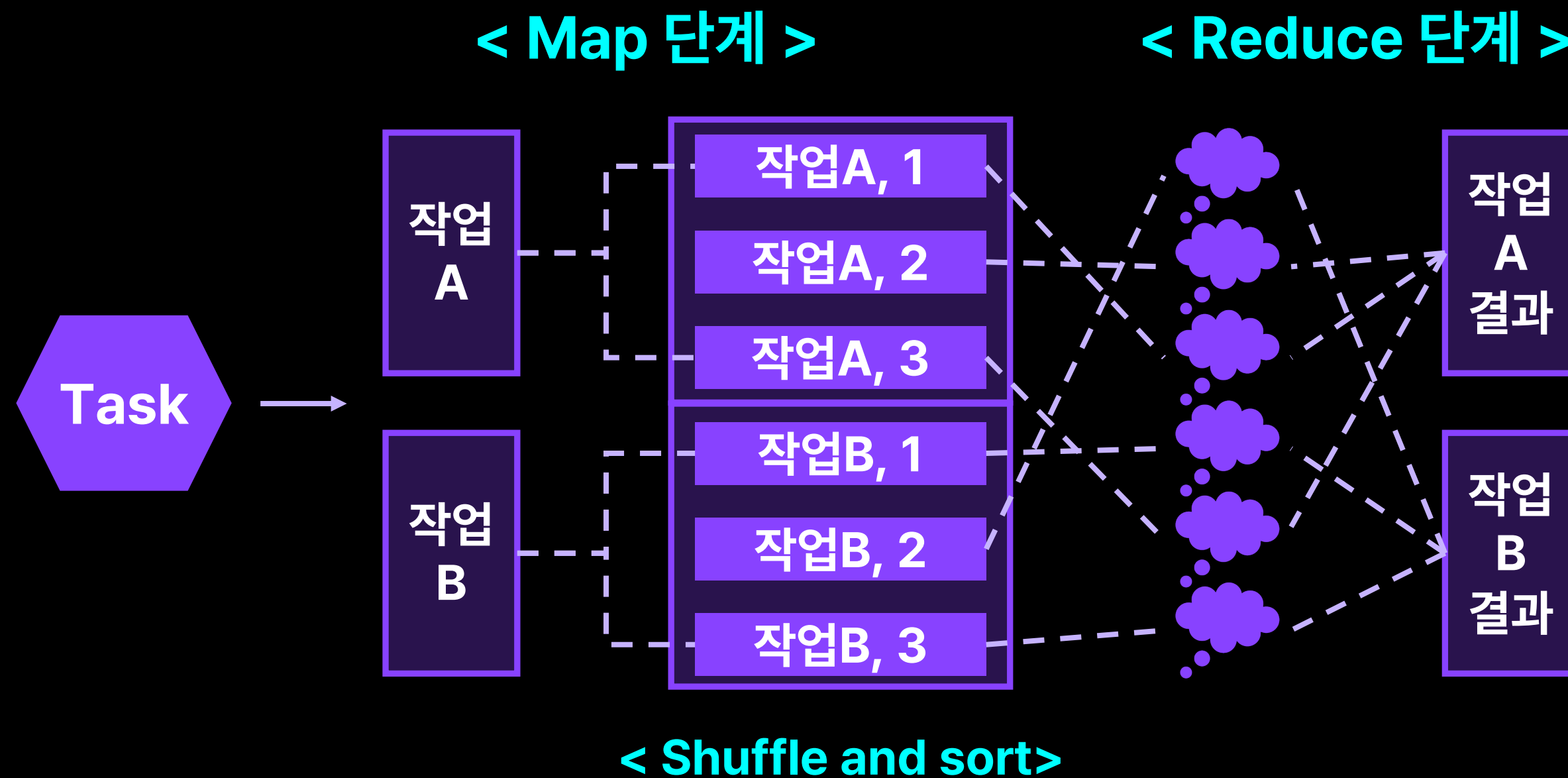
[재사용 가능]

- 유사한 task를 처리해야 하는 경우, 사전에 정의된 subgraph를 재활용 할 수 있습니다

[관리의 용이성]

- 타인 또는 미래의 자신이 쉽게 이해할 수 있습니다
- 디버깅이 쉬워집니다

4. 병렬 실행 노드



Map reduce는 분산 컴퓨팅의 방법 중 하나로,
하나의 task를 여러 작업을 나누고 작업한 후,
최종 결합하여 작업 결과를 취합



Map-reduce 브랜치를 사용하는 경우

[Why]

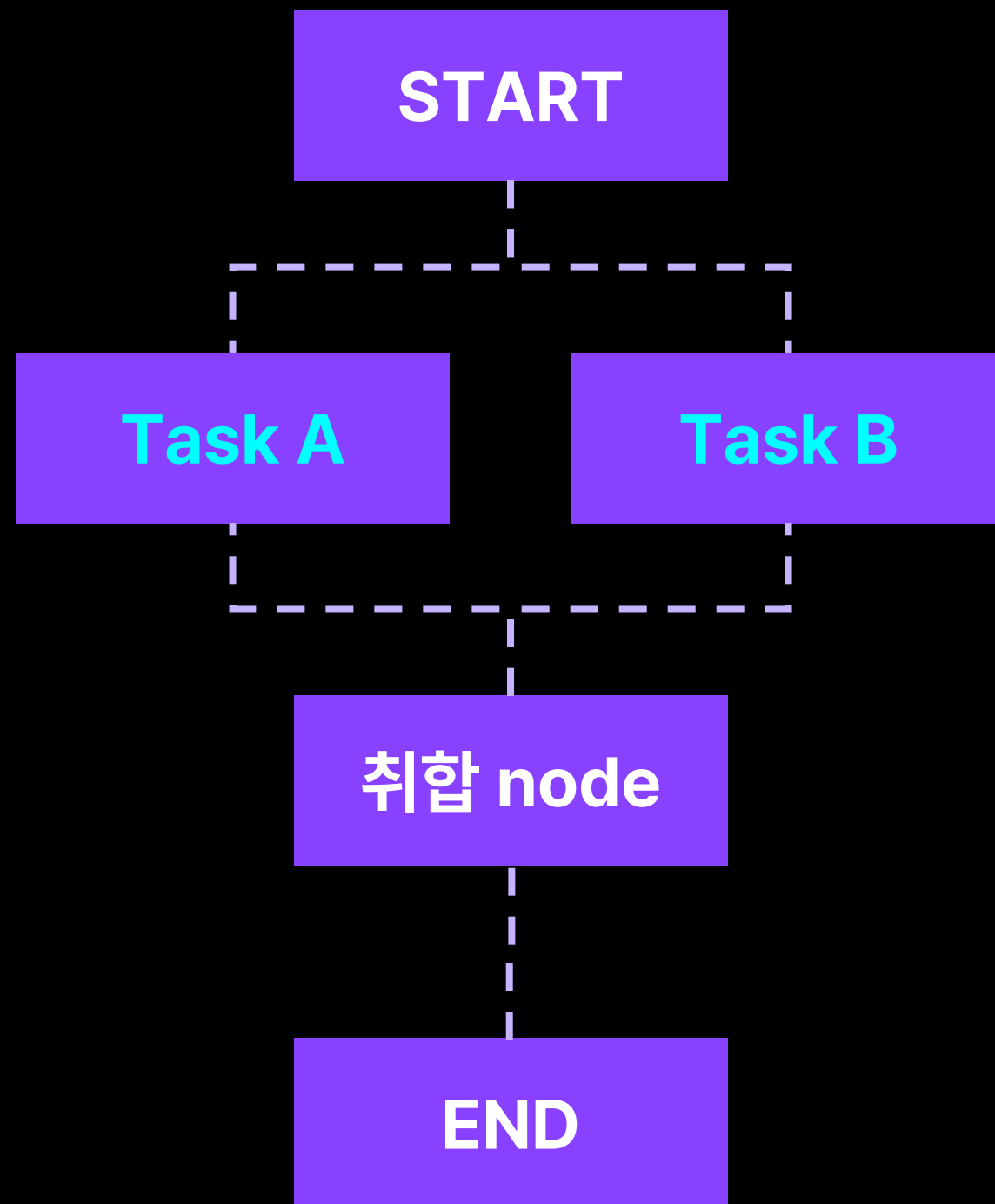
- Node execution time을 줄여야 함
- 하나의 task를 분할하여 효율적으로 수행

[What]

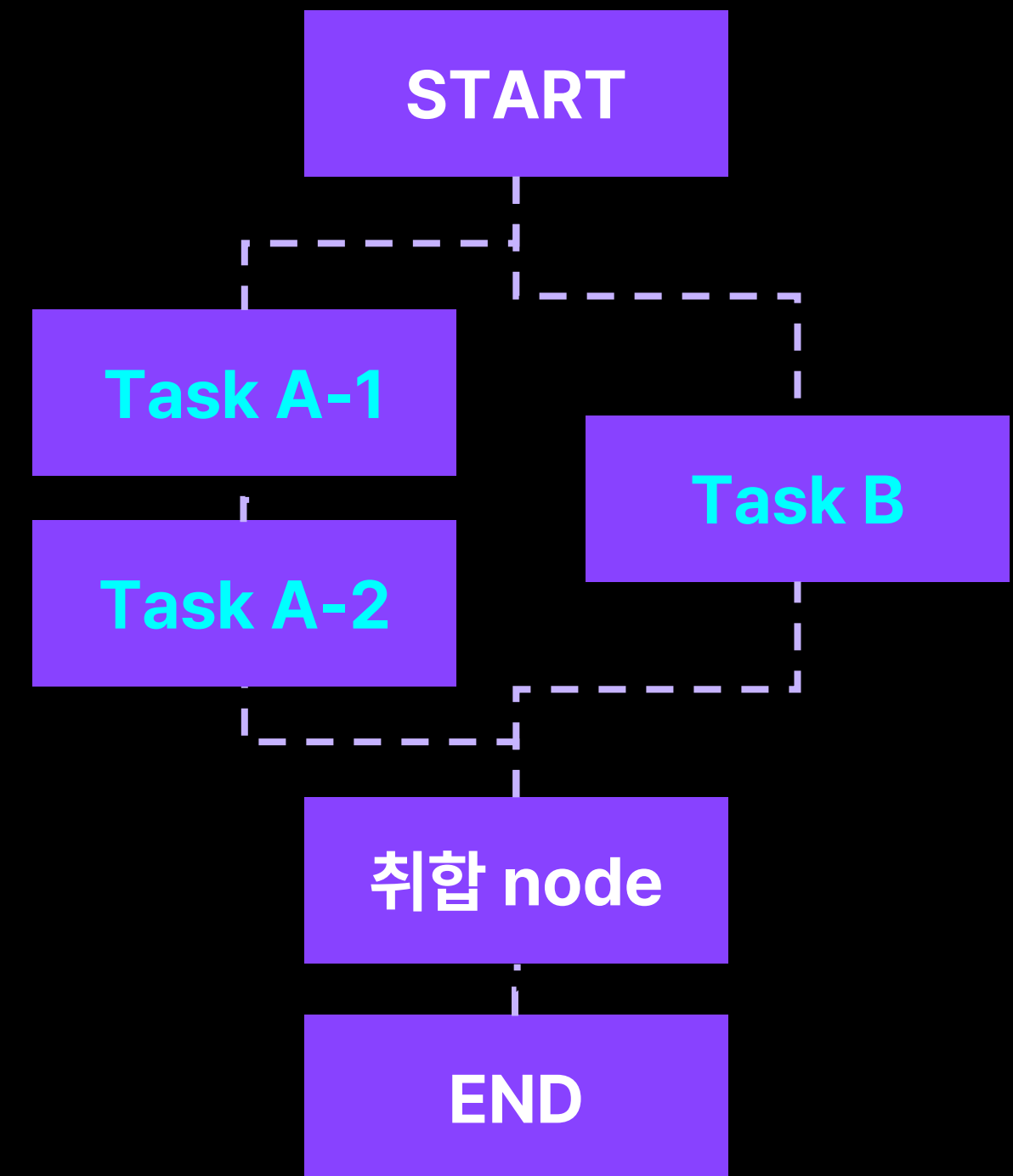
- 병렬 계산을 통해 작업을 빠르게 해야 하는 경우.
특히, 처리해야 하는 노드 및 단계가 많을 경우

[How]

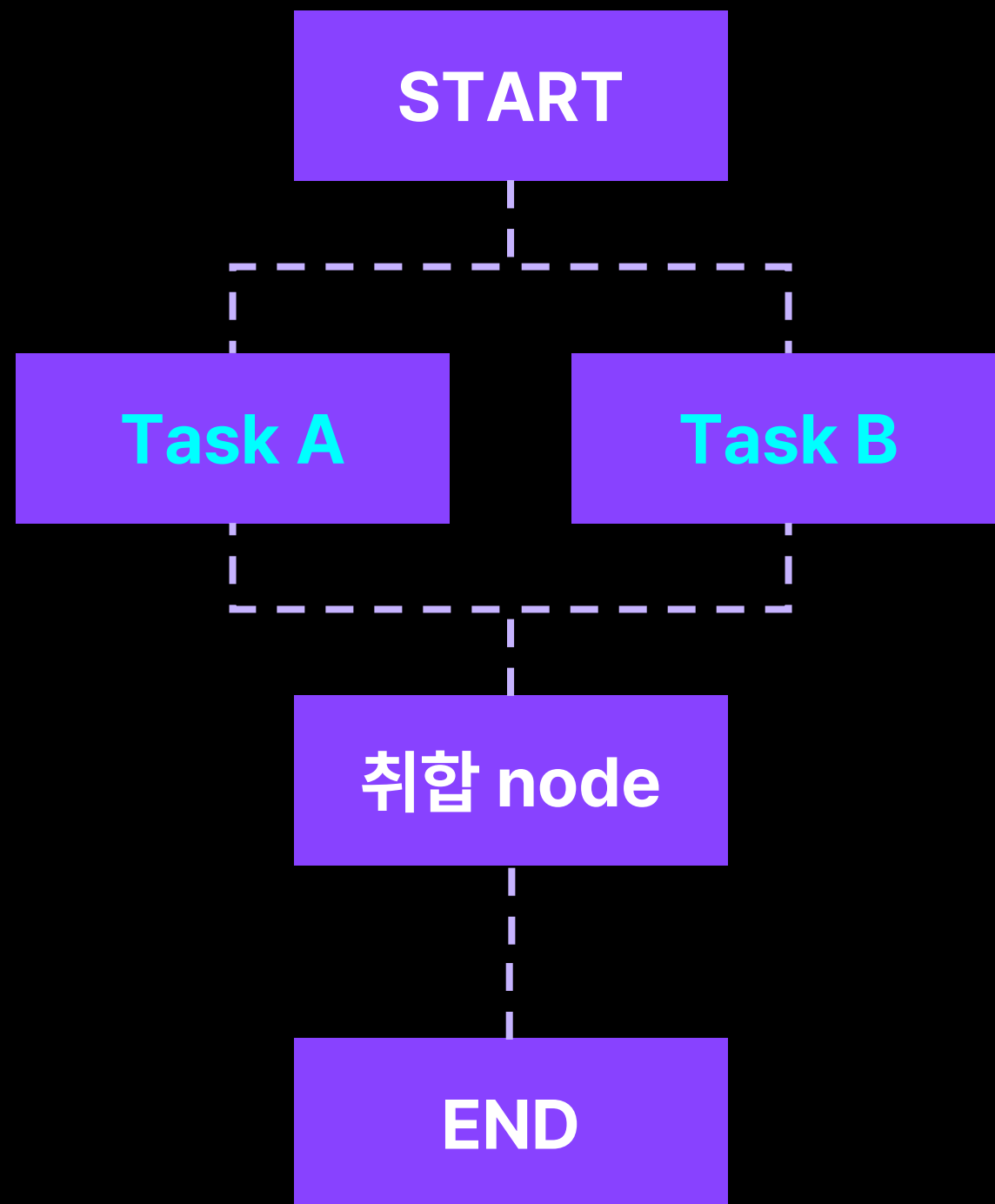
- 병렬 Node를 생성하여 각 작업을 독립적으로 완수
- 완성된 결과를 reduce하여 취합하는 node 생성



병렬 실행 예시 1



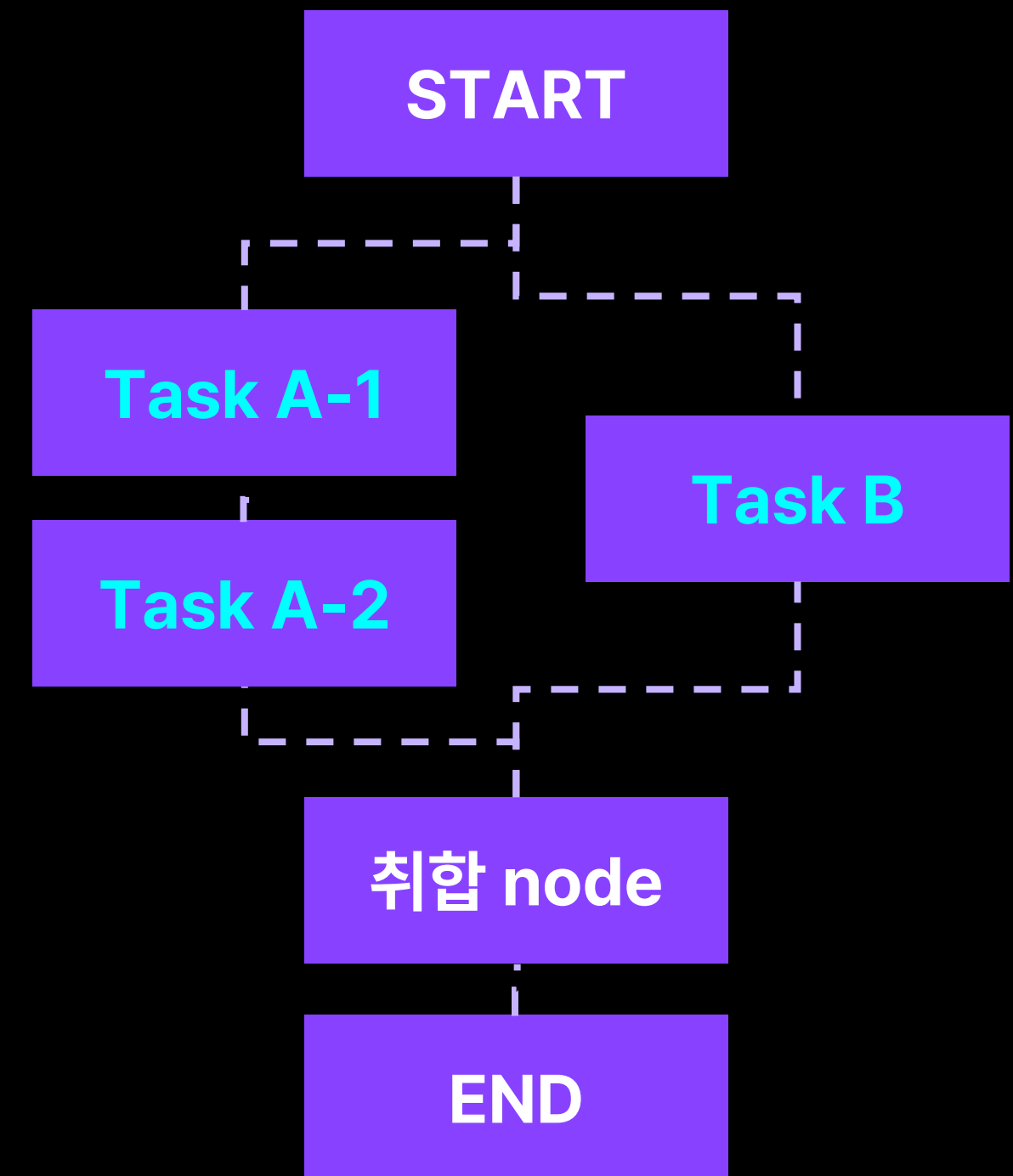
병렬 실행 예시 2



병렬 실행 예시 1

병렬 실행

- Map-reduce는 병렬 실행의 한 예시이며, fan-out, fan-in을 통해 처리
- Fan-in : 하나의 작업이 여러 하위 작업으로 분할되어 여러 프로세서나 노드에서 병렬로 처리
- Fan-out : 병렬로 수행된 여러 작업의 결과를 결합해 최종 출력을 생성
- Fan-in과 fan-out을 customize하여 실행 할 수 있음



병렬 실행 예시 2